```
n ← length(A)
total ← A[0]
for i in {1..n-1} do
  total ← total + A[i] + A[i-1]^2
```

A.4.2 While Loop

As mentioned, `while` loops are intended to be more generic; as such, they can be used to implement the same logic of `for` loops, included the examples in the previous section. Here it is an equivalent for loop to compute the sum of the elements in an array:

```
n ← length(A)
i ← 0
total ← 0
while i < n do
  total ← total + A[i]
  i ← i + 1
```

Clearly the `for-each` syntax is a way to express the same logic with less code (and also, as importantly, it encapsulates the logic of iterating through indices, making it less error-prone than the `while` loop above, where we have to initialize and increment `i` explicitly).

With `while` loops, however, we can also write conditions that it would be hard or impossible to express using `for` statements: